About Me

Internship

UnifiedMentor

Course

Cybersecurity Internship (6 Months)

Duration

01/11/2025 – April

Intern ID

UMID14102561484

Email

rank1aditya@gmail.com

GitHub Repo

github.com/enterlectury/payload-encoder

Project Summary

A Python framework to encode and obfuscate test payloads, then simulate signature-based detection to measure evasion effectiveness. Built for red team education and defensive research.

87.5%	7 / 8	8	4
Bypass Rate	Bypassed	Variants	Modules

Run command:

```
$ python3 payload_encoder.py
```

Source / Repo:

https://github.com/enterlectury/payload-encoder/

What is this project?

This project builds a Python-based framework that takes a raw payload (like a shell command), applies multiple encoding and obfuscation techniques, then tests whether a simulated signature-based scanner can detect it. It shows how attackers hide malicious content — and how defenders can improve detection.

A. Encoding Module

- Base64 encoding & decoding
- XOR encryption (user-defined key)
- ROT13 substitution cipher

B. Obfuscation Module

- Random character insertion
- Character splitting & concatenation
- Escape-sequence obfuscation

C. Evasion Testing

- Compare against signature list
- Measure detection success/failure
- Highlight effective techniques

D. Reporting Engine

- List encoded outputs
- Behavior comparison
- Evasion insights + bypass rate

Step-by-step execution flow

STEP 1

↓

Input Payload

Raw script, shellcode, or text string loaded into the framework.

STEP 2

↓

Encoder Selection

Choose Base64, XOR (key=42), ROT13, or apply multiple layers.

STEP 3

↓

Apply String Obfuscation

Generate obfuscated versions: reversed, char-split, hex escape, junk inserted.

STEP 4

↓

Evasion Testing

Compare each variant against 17 simulated signature checks. Record detection/bypass.

STEP 5

Output & Reporting

Store encoded payloads, calculate bypass rate, generate final report file.

Three encoding techniques applied to the payload

Base64

Converts payload bytes to printable ASCII. Uses Python's base64 library. decode_base64() restores original.

XOR (key=42)

XORs every character's ASCII code with key 42. Returns hex string. Same key decodes it back — XOR is self-inverse.

ROT13

Shifts each letter 13 positions using codecs.encode(payload, 'rot_13'). Applying ROT13 twice = original string.

```
┌──(SAM㉿Karan)-[~/UM]
└─$ python3 payload_encoder.py
============================================================
  Custom Payload Encoder & Obfuscation Framework
============================================================

[Step 1] Original Payload:
  powershell -nop -c exec(payload); /bin/sh reverse_shell
```

[Screenshot — Step 1 & Step 2 terminal output]

```
[Step 2] Encoding the payload...

  Base64 encoded:
  cG93ZXJzaGVsbCAtbm9wIC1jIGV4ZWMocGF5bG9hZCk7IC9iaW4vc2ggcmV2ZXJzZV9zaGVsbA==

  XOR encoded (key=42, shown as hex):
  5a455d4f5859424f46460a0744455a0a07490a4f524f49025a4b5346454b...

  ROT13 encoded:
  cbjrefuryy -abc -p rkrp(cnlybnq); /ova/fu erirefr_furyy
```

Four obfuscation techniques applied after encoding

Reverse String

Code: `payload[::-1]`

→ llehs_esrever hs/nib/ ;)daolyap(cexe...

ASCII Char Split

Code: `'-'.join(str(ord(c)) for c in p)`

→ 112-111-119-101-114-115-104-101...

Hex Escape Sequence

Code: `'\x{:02x}'.format(ord(c))`

→ \x70\x6f\x77\x65\x72\x73...

Random Junk Insertion

Code: insert |x| marker every 4 chars

→ powe|p|rshe|g|ll -|d|nop |m|...

[Screenshot — Step 3 terminal output]

```
[Step 3] Applying obfuscation techniques...
  Reversed string:
  llehs_esrever hs/nib/ ;)daolyap(cexe c- pon- llehsrewop
  ASCII char split (first 60 chars):
  112-111-119-101-114-115-104-101-108-108-32-45-110-111-112-32...
  Hex escape sequence (first 60 chars):
  \x70\x6f\x77\x65\x72\x73\x68\x65\x6c\x6c\x20\x2d\x6e\x6f\x70...
  Random junk inserted:
  powe|p|rshe|g|ll -|d|nop |m|-c e|p|xec(|a|pay|l|v|oad)|i|; /b...
```

Signature scanner vs. obfuscated variants

Method	Result
Original Payload	DETECTED
Base64 Encoded	BYPASSED
XOR Encoded	BYPASSED
ROT13 Encoded	BYPASSED
Reversed String	BYPASSED
ASCII Char Split	BYPASSED
Hex Escape Sequence	BYPASSED
Random Junk Inserted	BYPASSED
Bypass Rate: 87.5% — 7 out of 8 variants bypassed detection	

[Screenshot — Step 4 evasion test output]

```
[Step 4] Running evasion tests against signature list...

  [DETECTED] Original Payload
  [BYPASSED] Base64 Encoded
  [BYPASSED] XOR Encoded
  [BYPASSED] ROT13 Encoded
  [BYPASSED] Reversed String
  [BYPASSED] ASCII Char Split
  [BYPASSED] Hex Escape Sequence
  [BYPASSED] Random Junk Inserted

[Step 5] Generating report...

============================================================
  PAYLOAD OBFUSCATION FRAMEWORK - EVASION REPORT
  Date: 2026-04-22 09:56:44
============================================================
```

Auto-generated after every run

[Screenshot — evasion_report.txt full output]

```
============================================================
  PAYLOAD OBFUSCATION FRAMEWORK - EVASION REPORT
  Date: 2026-04-22 09:56:44
============================================================

Original Payload:
  powershell -nop -c exec(payload); /bin/sh reverse_shell

Summary:
  Total tested  : 8
  Detected      : 1
  Bypassed      : 7
  Bypass Rate   : 87.5%

------------------------------------------------------------

  Method  : Original Payload
  Result  : DETECTED
  Preview : powershell -nop -c exec(payload); /bin/sh reverse_shell
  Matched : /bin/sh, exec(, powershell, payload, reverse_shell

  Method  : Base64 Encoded
  Result  : BYPASSED
  Preview : cG93ZXJzaGVsbCAtbm9wIC1jIGV4ZWMocGF5bG9hZCk7IC9iaW4vc2ggcmV2ZXJzZV9zaGVsbA==

  Method  : XOR Encoded
  Result  : BYPASSED
  Preview : 5a455d4f5859424f46460a0744455a0a07490a4f524f49025a4b5346454b4e03110a054843440559

  Method  : ROT13 Encoded
  Result  : BYPASSED
  Preview : cbjrefuryy -abc -p rkrp(cnlybnq); /ova/fu erirefr_furyy

  Method  : Reversed String
  Result  : BYPASSED
  Preview : llehs_esrever hs/nib/ ;)daolyap(cexe c- pon- llehsrewop

  Method  : ASCII Char Split
  Result  : BYPASSED
  Preview : 112-111-119-101-114-115-104-101-108-108-32-45-110-111-112-32-45-99-32-101-120-10

  Method  : Hex Escape Sequence
  Result  : BYPASSED
  Preview : \x70\x6f\x77\x65\x72\x73\x68\x65\x6c\x6c\x20\x2d\x6e\x6f\x70\x20\x2d\x63\x20\x65

  Method  : Random Junk Inserted
  Result  : BYPASSED
  Preview : powe|p|rshe|g|ll -|d|nop |m|-c e|p|xec(|a|payl|v|oad)|i|; /b|r|in/s|n|h re|s|ver
```

Report contains:

Date & Time:	Timestamped on every run
Original Payload:	Stored at top of report
Total Tested:	Count of all variants run
Detected:	How many were caught
Bypassed:	How many evaded scanner
Bypass Rate:	Calculated as percentage
Per-Method:	Result + matched signatures
Saved to Disk:	Written to evasion_report.txt

Technologies used & what this project teaches

Tools & Libraries

Python 3	Primary programming language
base64	Base64 encode and decode
codecs	ROT13 cipher implementation
random, string	Random junk char generation
re	Regex to remove junk markers
datetime	Timestamp for report output

Learning Outcomes

- How payloads are manipulated for evasion
- How obfuscation complicates malware detection
- How red teams hide malicious content
- How blue teams strengthen detection rules
- Practical encoding and transformation logic
- Python scripting for security tools

Summary & Key Takeaway

87.5%	8	7 / 8	1 / 8
Bypass Rate	Variants Tested	Bypassed Detection	Detected

Key Takeaway

Signature-based detection alone is not enough. All 7 obfuscated variants bypassed a scanner that instantly caught the original payload. Real security tools need to layer signatures with behavioral analysis, heuristics, and sandboxing.

Defensive Insights

- Decode Base64 and hex sequences before signature scanning
- Write YARA rules that also match encoded/obfuscated patterns
- Combine signature scanning with behavioral analysis and sandboxing
- Red team knowledge directly improves blue team detection rules